

threads (Drivers) that stream data batches through execution pipelines.

- *Operators (exec/)*: Physical processing units performing table scans (TableScan), filtering (Filter), projections (Project), multi-threaded joins (HashJoin), aggregations (HashAgg), sorting (Sort), and disk spilling (Spiller).
- *Expression Eval (expression/)*: Compiles SQL math and string expressions into fast execution loops (compileExpressions()) with Common Subexpression Elimination (CSE).
- *Functions (functions/)*: Built-in library of scalar functions, aggregate functions (facebook::velox::exec::Aggregate), and window functions.
- *Serializers (serializers/)*: Encodes vector batches (PrestoVectorSerde, UnsafeRowSerializer) for ultra-fast network exchange between cluster nodes.

Layer 3: Core data structures and I/O

- *Vectors (vector/)*: Columnar memory representation. Data is packed in vertical columns (FlatVector, zero-copy DictionaryVector, deferred LazyVector), enabling CPUs to process hundreds of values per clock cycle using SIMD instructions.
- *Type system (type/)*: Manages primitive types and complex structures (arrays, maps, structs).
- *Connectors (connectors/)*: Read/write data from various data sources (HiveConnector, TpchConnector).
- *I/O — DWIO (dwio/)*: I/O file readers for Apache Parquet, ORC/DWRF, Meta's fast columnar format (Nimble), and Text.
- *Filter pushdown*: DWIO readers evaluate filters directly inside storage files, skipping non-matching row groups before downloading data into RAM.

Layer 4: Memory, cache and infrastructure

- *Memory management (common/memory/)*: Tracks RAM usage in a

strict hierarchy (MemoryPool). If a query approaches memory limits, MemoryReclaimer automatically spills temporary data to disk, preventing Out-Of-Memory (OOM) crashes.

- *File Cache (AsyncDataCache)*: Asynchronously caches hot metadata and file pages in RAM, avoiding operating system page-cache thrashing.
- *Buffers and arenas (buffer/)*: Manages contiguous memory blocks (Buffer, BufferView, StreamArena, MmapArena).
- *Common utilities (common/)*: Process logging, metrics (TraceContext), and configuration.

Layer 5: Storage and formats

Interfacing layer connecting Velox to cloud object stores (AWS S3, Azure ADLS, Google Cloud Storage), distributed filesystems (HDFS, Local Disk), and open table formats (Hive, Apache Iceberg, Delta Lake).

Layer 6: Experimental acceleration and exchange

- *GPU acceleration*: Offloads heavy vector compute to Nvidia GPUs using Wave (Velox CUDA framework) or RAPIDS cuDF.
- *Breeze*: Low-level SIMD vector primitives.
- *UCX exchange*: High-speed RDMA network shuffle plugin for ultra-fast server-to-server data transfers.

The journey of a query inside Velox

The journey of a query inside Velox is streamlined, highly vectorized, and occurs entirely on a single host. Because Velox is an execution library rather than a full database, it does not parse SQL or perform global optimisation. It sits strictly on the local data plane, executing a physical query plan that has already been fully optimised by a parent engine like Presto or Spark.

Here is the step-by-step journey of

how data moves through Velox.

Query hand-off and translation:

The parent engine's coordinator parses the query, optimises the plan, and hands a specific physical query plan fragment to Velox on the local host. Velox ingests this physical plan and maps it to a *Task* (the local unit of execution).

Task and Driver execution:

The task is divided into one or more *Pipelines*, each consisting of a sequence of operators that can process data continuously. Every pipeline is executed by one or more *Drivers*, which are Velox's execution objects scheduled on a shared worker thread pool. If a Driver becomes blocked, for example while waiting for I/O, it yields execution so that other Drivers can continue making progress, keeping CPU resources fully utilised.

 **Note:** Drivers are execution objects, not operating system threads. They are scheduled on worker threads managed by the host engine.

Pipelined scan and expression

evaluation: Once execution begins, data enters the pipeline through storage connectors and file readers.

Velox reads formats such as Parquet, ORC/DWRF, Nimble, and Text using its DWIO layer. During scanning, it applies optimisations such as column pruning and predicate pushdown, ensuring that only the columns and row groups required by the query are read from storage.

The data is represented internally as columnar vectors and streamed through the execution pipeline in batches. Expression evaluation -- including filters, projections, and arithmetic operations—is performed over these vector batches, allowing the engine to leverage SIMD instructions and cache-friendly memory access for significantly higher throughput than row-by-row execution.

Memory arbitration and spill

handling: As operators execute,